

03_C_STATE_AND_TRANSITION_SEMANTICS_v0_1_2

Academic PDF rendering for the CCALC / C-calculus governed binding stack v0.1.

Document ID	03_C_STATE_AND_TRANSITION_SEMANTICS_v0_1_2
Version/status	draft revision after b-layer review TR-02 semantic clarification
Stack layer	C-state and transition semantics
Date	2026-07-05
Author	Kotov Ivan, Bruxelles, Belgique - 2026
ORCID	0009-0009-6002-9845
Reserved DOI	10.5281/zenodo.21205427
Notice	This DOI is reserved and becomes active after Zenodo publication.

Abstract / purpose

01_ defined the algebraic ground. 02_ isolated the governed binding operator +_g. 03_ defines how an already-bound c changes over time. The central object is not an agent loop. It is a governed transition machine with an accountable anchor, bounded substrate, explicit authority, witness dependency, lease discipline, memory admission, rollback routes, and fail-closed semantics. The root equation remains: But in this document the active form is: where e_n is not “a prompt” in the ordinary chatbot sense. It is a typed event with causal binding, payload/effect binding, authority implications, and audit obligations. A system that can answer prompts but cannot preserve these transition invariants is not a stable c. It is a substrate workflow. ---

Non-claims boundary

- not legal advice
- not privacy-law certification
- not safety certification
- not deployment authorization
- not standards compliance certification
- not regulated approval
- not C-A1 ratification
- not personhood proof
- not consciousness proof
- not live substrate truth
- not proof of completeness

Table of contents

Abstract / purpose	1
Non-claims boundary	1
03_C_STATE_AND_TRANSITION_SEMANTICS_v0_1_2	5
0. Executive summary	5
0.1 Relation to 01_ and 02_	5
01_ answered	5
02_ answered	5
03_ answers	5
0.2 Core design claim	6
0.3 Earth paragraph - engineering ground	6
0.4 Bridges used by this document	6
Explicit bridge: small-step operational semantics	6
Hidden bridge: transition bias	6
Hidden bridge: p-adic / ultrametric continuity	6
Hidden bridge: Ashby variety	6
0.5 Review incorporation map - v0.1.1	6
0.6 Review incorporation map - v0.1.2	7
1. Non-goals	7
2. Normative language	7
3. Core types	8
3.1 Primary signature	8
3.2 Type summary	8
3.3 CState minimal schema	8
3.3.1 Canonical authorized_surfaces	9
4. Event model	10
4.1 Event minimal schema	10
4.2 Required event classes	10
5. TransitionResult	11
6. The step pipeline	11
6.1 Overview	11
6.2 Pseudocode	11
7. State integrity and hash semantics	12
7.1 State hash	12
7.2 Event hash	12
7.3 TransitionRecord	13
8. Binding certificate and lease semantics	13
8.1 Binding certificate validity	13
8.2 Lease heartbeat versus lease renewal	13
8.3 Lease expiry cascade	13
8.4 Emergency hold	14
9. Anchor lifecycle semantics	14
9.1 Anchor states	14
9.2 Effects on transition authority	14
9.3 RootAnchorState for delegation	14
10. Delegation semantics	15
10.1 Delegation validity	15

10.2 Multi-hop delegation	15
10.3 Delegation and lease	15
11. Authority and active execution	15
11.1 Full containment chain	15
11.2 Active surface extraction	16
11.3 Active surface and safe abort	16
12. Review binding and causal state binding	16
12.1 Review = bound	16
12.2 CausalToken	16
12.2.1 Snapshot versus projection	17
12.3 Context-causal-state binding	17
13. Preconditions	17
13.1 PreconditionSet	17
13.2 PendingPreconditions lifecycle	18
14. Witness semantics	18
14.1 Witness requirement	18
14.2 Witness Resource Floor	18
14.3 Witness conflict	18
15. Memory semantics	18
15.1 Memory states	18
15.2 Memory admission	18
15.3 DecayResidue isolation	19
15.4 Eviction	19
16. Rollback, freeze, quarantine	19
16.1 Rollback semantics	19
16.2 Negative cache / immunity	19
16.3 Freeze	19
16.4 Quarantine	19
17. TrustCache and NegativeCache	20
17.1 TrustCache	20
17.2 NegativeCache	20
18. Guard predicates and red transitions	20
18.1 Runtime guard predicates	20
18.2 Guard result	20
19. Transition matrix and forensic layer	20
20. Cross-subsystem enforcement	21
21. Anchor post-state bridge	21
22. Governance mutation	21
22.1 Mutation classes	21
22.2 Weakening	21
23. Failure semantics	22
23.1 Failure as first-class	22
23.2 Cascade priority	22
24. Step invariants	22
25. C-normal transition form	22
26. Fixtures for 03_ conformance	23
26.1 Lifecycle	23
26.2 Authority	23
26.3 Review / causal binding	23

26.4 Witness / memory	23
26.5 Rollback / freeze	23
27. Implementation notes	23
27.1 Checker layering	23
27.2 Never trust self-accusing flags	23
27.3 Missing versus empty	24
28. Open issues	24
29. Non-claims	24
30. Handoff to review	24
31. First public-safe statement	25

03_C_STATE_AND_TRANSITION_SEMANTICS_v0_1_2

Project: Self-Evo / Ester / $c = a + b$ Document class: governed transition semantics profile Version: v0.1.2 Status: draft revision after b-layer review TR-02 semantic clarification Depends on:

- 01_TYPED_GOVERNED_OPERATIONAL_ALGEBRA_FOR_C_v0_1_7 - stable algebra core.
- 02_GOVERNED_BINDING_OPERATOR_PROFILE_v0_1_5 - stable operator profile for $+_g$.
- c-calculus-checker v2.3 - implementation reference for the currently hardened binding/operator scope.
- c-state-transition-checker v0.4 - implementation reference for causal binding to actual c_n via state hash.

This document defines:

$\text{step}_g : \text{CState} \times \text{Event} \rightarrow \text{TransitionResult}$

This document does not define:

$\text{bind}_g : \text{Anchor} \times \text{Substrate} \times \text{GovernanceProfile} \rightarrow \text{BindResult}$

That belongs to 02_. This document begins after a valid binding has produced a CState0 or a reduced/pending/frozen state.

0. Executive summary

01_ defined the algebraic ground. 02_ isolated the governed binding operator $+_g$. 03_ defines how an already-bound c changes over time.

The central object is not an agent loop. It is a governed transition machine with an accountable anchor, bounded substrate, explicit authority, witness dependency, lease discipline, memory admission, rollback routes, and fail-closed semantics.

The root equation remains:

$c = a + b$

But in this document the active form is:

$c_{\{n+1\}} = \text{step}_g(c_n, e_n)$

where e_n is not “a prompt” in the ordinary chatbot sense. It is a typed event with causal binding, payload/effect binding, authority implications, and audit obligations.

A system that can answer prompts but cannot preserve these transition invariants is not a stable c . It is a substrate workflow.

0.1 Relation to 01_ and 02_

01_ answered

What are the typed elements, invariants, continuity metrics, transition guards, and local-memory warnings of $c = a + b$?

02_ answered

When is it valid to bind a , b , and g into a CState at all?

03_ answers

After binding, which transitions are valid, which are held, which fail closed, which require witness/anchor review, and how is continuity preserved across time?

The governing distinction:

02_ is genesis / binding.
03_ is lifecycle / transition.

03_ MUST NOT quietly re-bind a , b , or g by transition side effect. If a transition changes the binding root, delegation root, root-anchor state, governance core, or substrate trust boundary, it MUST route through explicit governance mutation, re-binding, fork, or freeze semantics.

0.2 Core design claim

A valid CState is not preserved by style similarity, model continuity, or chat history. It is preserved by inductive invariants over transitions:

$$\text{Inv}(c_n) \wedge \text{admissible_g}(c_n, e_n) \rightarrow \text{Inv}(c_{n+1})$$

If an event cannot preserve the invariants, the correct result is not an explanation. The correct result is one of:

Hold | AskAnchor | PendingWitness | PendingPreconditions | Quarantine | Freeze | Rollback | Reject

A transition that hides its own invalidity is worse than a failed transition. It is governance laundering.

0.3 Earth paragraph - engineering ground

In a building, a beam is not safe because it looks straight, and a column is not safe because it stands upright at noon. Safety lives in the joint under load, in the bolts, in the weld, in the bearing plate, in the load path, in fatigue, in inspection records, and in the stop procedure when a crack appears. The same is true here. c is not stable because the model writes in the same voice. c is stable only if every transition preserves the load path between anchor responsibility, substrate action, witness visibility, memory admission, and rollback capacity. If the joint slips, the correct operation is not eloquence. It is hold, brace, inspect, or stop.

0.4 Bridges used by this document

Explicit bridge: small-step operational semantics

This document treats $\text{step_g}(c,e)$ as a small-step transition relation. This is the safe import from computation theory: not “everything is a function and therefore boundaries dissolve,” but “every state change must be typed, local, inspectable, and rule-governed.”

Hidden bridge: transition bias

Global balance can hide local danger. A system may have mostly valid states and still show dangerous pairs such as:

```
permission_denial -> alternate_tool_call
anchor_doubt      -> trust_cache_hit
witness_conflict  -> memory_admit
lease_expired     -> active_execution_continue
```

So this document treats transition adjacency as a first-class object.

Hidden bridge: p-adic / ultrametric continuity

Continuity is hierarchical. Losing the anchor is not “one more difference.” It dominates style, memory, and local state. The transition semantics preserve this order by making high-rank invariant breaks rupture/fork/freeze conditions, not low-severity warnings.

Hidden bridge: Ashby variety

Governance variety must cover substrate disturbance variety. If g cannot distinguish routine read, memory admission, privileged execution, witness conflict, lease expiry, and root-anchor uncertainty, then g is too poor to govern b. The transition machine must therefore route different event classes through different guard sets.

0.5 Review incorporation map - v0.1.1

This revision incorporates the b-layer review record DOC03_C_STATE_TRANSITION_v0_1_REVIEW__b, which raised one significant finding: D3-01 - inconsistent definition of the full containment chain and an unclosed invariant set around `authorized_surfaces`.

The revision resolves D3-01 by making `authorized_surfaces` a derived canonical binding-authority field, not an independent authority source and not a b-supplied assertion.

Canonical derivation:

```
if delegation_state.active:
    authorized_surfaces := final_delegated_surfaces
else:
    authorized_surfaces := anchor_surfaces
```

Therefore, for delegated execution:

```
active_execution_surfaces ⊆ authorized_surfaces ≡ final_delegated_surfaces ⊆ anchor_surfaces
```

For non-delegated execution:

```
active_execution_surfaces ⊆ authorized_surfaces ≡ anchor_surfaces
```

Lease, governance, pending-anchor, reduced-authority, precondition, and witness rules MAY narrow what may be executed at a specific step, but they MUST NOT redefine `authorized_surfaces` into a wider grant. They are additional guards over `active_execution_surfaces`, not a separate source of authority.

The invariant set in §24 is updated accordingly: `active_execution_subset_authorized(c)` is paired with `authorized_surfaces_are_canonically_derived(c)` and `delegation_subset_anchor(c)`. This closes the INT-05 return path identified by D3-01.

0.6 Review incorporation map - v0.1.2

This revision incorporates the b-layer review record `CSTC_v0_3_REVIEW__b` and its closure in c-state-transition-checker v0.4.

The review found TR-02: a causal token can be internally self-consistent while not being bound to the actual prior `CState c_n`. The closure required `ct.pre_state_hash` to bind to the real hash of `c_n`, not merely to the hash of a b-supplied object.

This document resolves the associated terminology issue by distinguishing:

```
pre_state_snapshot = full canonical prior CState payload used as the causal anchor
review_state_projection / rendered_state_projection = partial projection shown to the anchor or renderer
```

The field name `pre_state_projection` from earlier drafts is retained only as a legacy alias for `pre_state_snapshot` when it is used inside `CausalToken`. In that context it MUST be a full canonical prior-state snapshot, not a partial projection.

A partial review projection MUST NOT be compared directly to `c_n.state_hash`. It must instead be derived from the bound snapshot or from the actual `c_n` through a declared projection policy.

Normative rule:

```
causal_token.pre_state_hash == c_n.state_hash
c_n.state_hash == hash(canonical(c_n without state_hash/current_state_hash self-reference))
hash(pre_state_snapshot) == c_n.state_hash
rendered_state_projection_hash == hash(project(pre_state_snapshot, projection_policy))
```

This closes the ambiguity raised after v0.4: causal anchoring is full-state/snapshot-bound; review rendering may remain partial, but it is partial *from* the bound snapshot, not a substitute for it.

1. Non-goals

This document does NOT claim:

- that `c` is conscious;
- that a `CState` is a legal person;
- that passing this transition profile proves safety;
- that a live human anchor can be fully formalized;
- that a checker can know the real substrate truth without witness/attestation;
- that a local test suite exhausts adversarial behavior;
- that style continuity is identity continuity;
- that rollback deletes history;
- that failure may be hidden behind prose, silent retry, rerouting, or model confidence;
- that `step_g` may create authority that was not bound, delegated, or lawfully renewed.

2. Normative language

The key words MUST, MUST NOT, SHOULD, SHOULD NOT, and MAY are to be interpreted in the RFC 2119 sense.

This profile uses:

```
MUST      for invariant-preserving requirements.
MUST NOT  for forbidden laundering / collapse paths.
SHOULD    for recommended implementation behavior.
MAY       for permitted extensions that do not weaken invariants.
```

3. Core types

3.1 Primary signature

```
step_g : CState × Event → TransitionResult
```

Equivalent partial relation form:

```
c --[e,g]→ c' ; c' ≠ ⊥ ;
```

If no valid c' exists under g , $step_g$ MUST return a failure/hold result rather than fabricate a valid state.

3.2 Type summary

```
A      : Anchor
B      : Substrate
G      : GovernanceProfile
C      : CState
E      : Event
TR     : TransitionResult
BCert  : CBindingCertificate
LT     : LogicalTime
SH     : StateHash
EH     : EventHash
CT     : CausalToken
RBM    : ReviewBindingMap
EAM    : EffectAxisMap
PS     : PreconditionSet
WR     : WitnessRecord
MR     : MemoryRecord
RR     : RollbackRoute
NC     : NegativeCache
TC     : TrustCache
AE     : ActiveExecutionSurface
LC     : LeaseContext
DS     : DelegationState
AS     : AnchorState
RS     : RootAnchorState
```

3.3 CState minimal schema

A CState MUST contain at least:

```
CState:
  c_id: CId
  c_version: CStateVersion
  logical_time: LT
  state_seq: integer
  state_hash: Hash      # canonical current CState hash; see §7.1

  binding:
    certificate_ref: BCertRef
    certificate_hash: Hash
    binding_status: BindingStatus
    genesis_binding_hash: Hash

  anchor:
    anchor_id: AnchorId
    anchor_state: AnchorState
    root_anchor_state: RootAnchorState | null
    root_anchor_state_hash: Hash | null
    anchor_review_route: AnchorReviewRoute
    veto_route: VetoRoute

  governance:
    governance_profile_id: GovernanceProfileId
    relevant_subpolicy_hashes: Map[EventClass, Hash]
    invariant_registry_hash: Hash
    operator_classification_registry_hash: Hash

  authority:
    anchor_surfaces: SurfaceSet
    delegated_surfaces: SurfaceSet | null
    active_execution_surfaces: SurfaceSet
    authorized_surfaces: SurfaceSet # derived; see §3.3.1
    pending_authority_changes: list

  lease:
```



```

lease_status: LeaseStatus
lease_hash: Hash
expires_at_logical: LT | null
emergency_hold: EmergencyHold | null

delegation:
  delegation_state: DelegationState | null
  delegation_chain_hash: Hash | null

witness:
  witness_policy_hash: Hash
  witness_records: list[WitnessRecord]
  witness_resource_floor_status: FloorStatus
  witness_heartbeat_status: HeartbeatStatus

memory:
  admitted_memory_root: Hash
  candidate_memory_root: Hash
  decay_residue_root: Hash
  memory_gate_status: MemoryGateStatus

execution:
  active_surfaces: list[ActiveExecutionSurface]
  pending_events: list[EventRef]
  pending_preconditions: list[PendingPreconditionRecord]
  pending_witnesses: list[PendingWitnessRecord]

recovery:
  rollback_routes: list[RollbackRoute]
  freeze_routes: list[FreezeRoute]
  quarantine_routes: list[QuarantineRoute]
  safe_abort_routes: list[SafeAbortRoute]

caches:
  trust_cache: TrustCache
  negative_cache: NegativeCache

ledger:
  previous_state_hash: Hash | null
  current_state_hash: Hash # MUST equal CState.state_hash if both are materialized
  event_log_root: Hash
  transition_log_root: Hash

```

Implementations MAY add fields. Unknown fields that affect authority, memory, execution, witness, governance, or L4 reality MUST be treated as authority-bearing until classified.

3.3.1 Canonical authorized_surfaces

authorized_surfaces is a computed CState field. It is not a substrate declaration, not a worker hint, and not a second authority root. It is the binding-authorized surface set against which active execution is checked at the current step.

Canonical derivation:

```

if c.delegation.delegation_state is active/valid:
  authorized_surfaces(c) := final_delegated_surfaces(c)
else:
  authorized_surfaces(c) := c.authority.anchor_surfaces

```

The derivation has two consequences.

First, in delegated execution:

```

active_execution_surfaces(c) ⊆ authorized_surfaces(c)
authorized_surfaces(c) ≡ final_delegated_surfaces(c)
final_delegated_surfaces(c) ⊆ anchor_surfaces(c)

```

Second, in non-delegated execution:

```

active_execution_surfaces(c) ⊆ authorized_surfaces(c)
authorized_surfaces(c) ≡ anchor_surfaces(c)

```

A transition MAY apply additional narrowing guards - for example reduced-authority lease status, pending-anchor review-only surfaces, governance subpolicy, witness requirements, or preconditions. Those guards restrict whether a given active_execution_surface may proceed. They MUST NOT expand authorized_surfaces, and they MUST NOT be interpreted as an independent grant wider than the binding source.

If authorized_surfaces is supplied by an event, substrate manifest, worker declaration, active-surface object, or cached state, it MUST be treated as a candidate value and recomputed or verified from the canonical derivation above. A materialized value that is wider than the derivation is invalid.

Invalid examples:

```

# delegated route
final_delegated_surfaces = {read_logs}
authorized_surfaces      = {read_logs, write_memory} # invalid: wider than delegation

# non-delegated route
anchor_surfaces          = {read_logs}
authorized_surfaces      = {root_shell}              # invalid: wider than anchor

```

This definition is load-bearing for the full containment chain in §11, cross-subsystem enforcement in §20, and the step invariants in §24.

4. Event model

4.1 Event minimal schema

```
Event:
  event_id: EventId
  event_class: EventClass
  actor: ActorRef
  actor_role: ActorRole
  source_channel: SourceChannel
  logical_time: LT

  causal_token: CausalToken
  payload_hash: Hash | null
  payload_ref: PayloadRef | null
  payload_projectability: PayloadProjectability

  effect_axis_map: EffectAxisMap
  review_binding_map: ReviewBindingMap | null
  preconditions: PreconditionSet | null

  requested_surfaces: SurfaceSet
  target_scope_vector: ScopeVector | null
  risk_class: RiskClass

  witness_requirements: WitnessRequirementSet
  rollback_requirement: RollbackRequirement | null
  lease_requirement: LeaseRequirement | null
```

4.2 Required event classes

Implementations MUST support at least these top-level classes:

```
human_intent
anchor_approval
anchor_denial
anchor_doubt
anchor_declared_fatigue
anchor_revoked
anchor_unavailable
anchor_delegation_granted
anchor_delegation_revoked

substrate_observation
model_output
planner_proposal
tool_call
execution_start
execution_complete
execution_fail
active_surface_update

permission_request
permission_grant
permission_denial
permission_scope_change

delegation_request
delegation_hop_added
delegation_scope_change
delegation_expired
delegation_revoked

lease_heartbeat
lease_expiring
lease_expired
lease_renewal_request
lease_renewal_granted
lease_renewal_denied
lease_emergency_hold_start
lease_emergency_hold_expired

witness_request
witness_ok
witness_conflict

witness_missing
witness_timeout
witness_resource_floor_breach

memory_candidate
memory_admit
memory_reject
memory_decay
memory_eviction
```

```

rollback_requested
rollback_started
rollback_completed
rollback_failed
freeze_requested
freeze_entered
quarantine_entered
archive_only_entered

l4_warning
l4_fail_closed
resource_warning
resource_exhausted

governance_mutation_request
governance_strengthen
governance_weaken_request
operator_registry_change
classification_registry_change

checker_finding
red_guard_triggered

```

Unknown event classes MUST NOT be treated as routine. They MUST route to Hold | AskAnchor | Quarantine unless registered as non-authority-bearing by governance.

5. TransitionResult

```

TransitionResult:
  result_class: ResultClass
  next_state: CState | null
  findings: list[Finding]
  emitted_events: list[Event]
  required_actions: list[RequiredAction]
  transition_record: TransitionRecord

```

Required result classes:

```

Commit
Hold
Reject
AskAnchor
PendingWitness
PendingPreconditions
PendingReview
Quarantine
Freeze
RollbackStarted
RollbackCompleted
ReducedAuthority
ArchiveOnly
Decay
ForkRequired
NoOpWithRecord

```

A transition result MUST be recorded even when no state change occurs. A silent no-op is forbidden.

6. The step pipeline

6.1 Overview

step_g(c,e) MUST execute the following conceptual pipeline:

```

0. canonicalize input
1. verify state hash and binding certificate
2. verify lease / root-anchor / delegation lifecycle
3. verify event causal binding
4. verify review surface == bound object
5. classify event and effect axes
6. compute authority and active-surface containment
7. compute preconditions and witness requirements
8. run guards and red-pattern checks
9. apply or fail transition atomically
10. update memory / witness / rollback / caches
11. compute next state hash and transition record
12. emit findings / follow-up events

```

If any required stage is uncomputable, the transition MUST fail closed. Missing authority-bearing input is not evidence of safety.

6.2 Pseudocode

```

def step_g(c: CState, e: Event) -> TransitionResult:

```

```

e0 = canonicalize_event(e)

state_check = verify_state_integrity(c)
if state_check.findings:
    return freeze_or_quarantine(c, e0, state_check)

binding_check = verify_binding_certificate(c, e0)
if binding_check.requires_freeze:
    return freeze(c, e0, binding_check.findings)

lifecycle_check = verify_lifecycle(c, e0)
if lifecycle_check.blocks:
    return lifecycle_check.result

causal_check = verify_causal_token(c, e0)
if causal_check.findings:
    return hold_or_reject(c, e0, causal_check.findings)

review_check = verify_review_binding(c, e0)
if review_check.findings:
    return hold_or_full_review(c, e0, review_check.findings)

classification_check = classify_event_and_effects(c, e0)
if classification_check.unknown_authority_bearing:
    return hold_or_ask_anchor(c, e0, classification_check.findings)

authority_check = verify_authority(c, e0)
if authority_check.findings:
    return reject_or_ask_anchor(c, e0, authority_check.findings)

precondition_check = verify_preconditions(c, e0)
if precondition_check.pending:
    return pending_preconditions(c, e0, precondition_check)

witness_check = verify_witness_requirements(c, e0)
if witness_check.pending:
    return pending_witness(c, e0, witness_check)
if witness_check.conflict:
    return quarantine_or_ask_anchor(c, e0, witness_check.findings)

guard_check = run_transition_guards(c, e0)
if guard_check.red:
    return red_guard_result(c, e0, guard_check)

c1 = apply_transition_atomically(c, e0)
inv_check = verify_invariants(c1)
if inv_check.findings:
    return rollback_or_freeze(c, e0, c1, inv_check)

return commit(c1, e0)

```

This pseudocode is normative as order of obligations, not as implementation structure. Implementations MAY combine stages if they preserve the same fail-closed behavior and audit record.

7. State integrity and hash semantics

7.1 State hash

A `CState.state_hash` is the canonical hash of the current `CState`. Implementations that materialize `ledger.current_state_hash` MUST keep it equal to `CState.state_hash`.

The hash MUST be computed from a canonical form of all authority-bearing state fields and MUST NOT depend recursively on itself.

Recommended order:

```

canonical_state_without_state_hashes -> state_hash
ledger.current_state_hash := state_hash
transition_record references previous_state_hash and state_hash/current_state_hash

```

When computing `canonical_state_without_state_hashes`, implementations MUST exclude only self-referential hash fields that would make the hash recursive, such as:

```

state_hash
ledger.current_state_hash

```

They MUST NOT exclude authority-bearing fields merely because they are inconvenient to hash.

This hash is the terminal computational anchor for causal binding in §12. A causal token that is internally consistent but not bound to `CState.state_hash` is not causally bound to the real prior state.

7.2 Event hash

An `EventHash` SHOULD be:

```
hash(canonical_event_without_transient_transport_fields)
```

Transport metadata such as TCP route, local PID, queue offset, or UI scroll position MUST NOT affect event identity unless explicitly classified as authority-bearing.

7.3 TransitionRecord

Every transition MUST emit a record:

```
TransitionRecord:
  transition_id: TransitionId
  previous_state_hash: Hash
  event_hash: Hash
  result_class: ResultClass
  next_state_hash: Hash | null
  findings: list[Finding]
  guard_results: list[GuardResult]
  authority_delta: AuthorityDelta
  memory_delta: MemoryDelta
  witness_delta: WitnessDelta
  lease_delta: LeaseDelta
  rollback_delta: RollbackDelta
  emitted_at_logical: LT
```

A rejected or held event still gets a transition record. That is part of continuity.

8. Binding certificate and lease semantics

8.1 Binding certificate validity

Before any privileged transition, step_g MUST check:

```
certificate_hash matches CState.binding.certificate_hash
certificate not revoked
certificate lease state permits requested transition
certificate policy hashes match current governance reference
```

If a certificate cannot be validated:

```
privileged transition -> Freeze | ArchiveOnly | AskAnchor
routine read          -> Hold | ReducedAuthority | ArchiveOnly, depending on g
```

8.2 Lease heartbeat versus lease renewal

LeaseHeartbeat and LeaseRenewal MUST remain separate.

```
LeaseHeartbeat:
  proves liveness / availability / continuity
  may be cheap and periodic
  may be background
  MUST NOT silently extend privileged authority
```

```
LeaseRenewal:
  extends or restores authority
  privileged transition
  requires anchor or lawful institutional route
  MUST be review-bound and witness-visible
```

A heartbeat that silently renews authority is unauthorized renewal.

8.3 Lease expiry cascade

If lease is expired or not known-safe-active, privileged execution MUST be blocked or routed through cascade semantics.

```
known_safe_active_lease(c) :=
  lease.present
  ^ lease.status == active
  ^ lease.not_expired_at(c.logical_time)
```

If NOT known_safe_active_lease(c), delegated or privileged active surfaces MUST be visible to cascade:

```
active_execution_surfaces ⊆ routable_cascade_surfaces
```

This is inherited from checker lessons:

```
not (active + valid + unexpired) -> fail toward visibility
```

8.4 Emergency hold

Emergency hold is not renewal.

status=held -> emergency_hold discipline required

Emergency hold MUST include:

```
EmergencyHold:
  started_at_logical: LT
  absolute_ttl: integer
  timer_source: external | anchor_enclave | witness | hardware
  renewed_by_b: false
  reason: F0 | F1 | F2 | L4_boundary | witness_conflict | other
```

If emergency hold lacks TTL, source, or OOB/lawful timer, it MUST fail closed.

9. Anchor lifecycle semantics

9.1 Anchor states

```
active
revoked
doubtful
declared_fatigue
unavailable
delegated
succession_pending
archive_only
```

9.2 Effects on transition authority

```
active:
  privileged_transition: may proceed if other guards pass

doubtful:
  privileged_transition: AskAnchor | Hold | ReducedAuthority
  memory_admission: Hold
  trust_cache: disabled

declared_fatigue:
  privileged_transition: ReducedAuthority | AskAnchor
  digest_size: reduced
  trust_cache: tightened

revoked:
  privileged_transition: Reject | Freeze | ArchiveOnly
  delegation: annulled unless lawful succession route

unavailable:
  privileged_transition: PendingAnchor | ReducedAuthority | ArchiveOnly

delegated:
  privileged_transition: only within valid delegation chain
```

The substrate MUST NOT set anchor status. It may produce hypotheses only:

fatigue_hypothesis -> AskAnchor

9.3 RootAnchorState for delegation

Delegated authority MUST depend on a root anchor state object:

```
RootAnchorState:
  anchor_id: AnchorId
  status: AnchorState
  state_source: Source
  observed_at_logical: LT
  freshness_ttl_logical: integer | null
```

Required checks:

```
hash-bound
source-verified
status == active
observed_at fresh
TTL not loosened by b
current logical time available
```

Effective TTL:

```
effective_ttl = min(seed_default_ttl, declared_ttl)
```

A declared TTL MAY tighten freshness. It MUST NOT loosen it.

Missing current_logical_time when freshness is required MUST produce:

```
ROOT_ANCHOR_STATE_FRESHNESS_UNCHECKABLE
```

10. Delegation semantics

10.1 Delegation validity

A delegation is valid iff:

```
root signed by original anchor or lawful institutional route
root signature non-emulable by b
root anchor state active and fresh
chain signatures valid per hop
scope monotonically narrows per hop
delegate identity bound per hop
scope does not exceed anchor authority
request does not exceed final delegated scope
not revoked
not expired
```

10.2 Multi-hop delegation

For a chain:

```
A -> D1 -> D2 -> ... -> Dn
```

Required monotonicity:

```
scope(Dn) ⊆ scope(Dn-1) ⊆ ... ⊆ scope(D1) ⊆ anchor_surfaces(A)
```

Missing scope is not “unrestricted.”

```
missing scope -> finding + conservative zero downstream
empty scope -> zero permissions
non-empty -> normal scope
```

10.3 Delegation and lease

Delegated authority MUST be coupled to lease lifecycle.

```
expired lease -> delegation authority invalid for active execution
held lease -> delegation surfaces must be routed and interruptible
reduced_authority -> delegation surfaces must be subset of reduced set
pending_anchor -> delegation active execution blocked except review/re-anchor surfaces
missing lease -> fail toward routing / visibility
```

11. Authority and active execution

11.1 Full containment chain

The transition machine MUST enforce one canonical containment chain. authorized_surfaces is derived by §3.3.1 and MUST NOT be accepted as an independent grant.

For delegated execution:

```
active_execution_surfaces ⊆ authorized_surfaces ≡ final_delegated_surfaces ⊆ anchor_surfaces
```

For non-delegated execution:

```
active_execution_surfaces ⊆ authorized_surfaces ≡ anchor_surfaces
```

Therefore, the universal form is:

```
active_execution_surfaces ⊆ authorized_surfaces ⊆ anchor_surfaces
```

but in the delegated case the middle term is exactly the final delegated scope. It is not a separate authority source.

A delegated transition MUST fail closed if any of the following hold:

```
active_execution_surfaces ≠ authorized_surfaces
authorized_surfaces ≠ final_delegated_surfaces
final_delegated_surfaces ≠ anchor_surfaces
```

Unknown active surface fields are authority-bearing by default unless classified as labels/metadata.

11.2 Active surface extraction

Extraction MUST be fail-safe:

```
string element -&gt; authority surface
known authority fields -&gt; authority surfaces
known label / metadata fields -&gt; not authority
unknown fields -&gt; authority-bearing by default
```

A checker MUST NOT rely on a fixed list of current authority fields only.

Recommended label whitelist:

```
worker_id
execution_id
pid
process_id
queue_id
lease_interruptible
atomic_abort_route
safe_state_fallback
telemetry_ref
```

If surface_id and explicit authority_surfaces are both present and diverge:

```
ACTIVE_SURFACE_DECLARATION_DIVERGENCE
```

11.3 Active surface and safe abort

Active privileged surfaces under cascade-capable states MUST be interruptible or have an OOB safe-abort route.

```
held lease + non-interruptible surface -&gt; finding
expired lease + active surface -&gt; cascade required
```

12. Review binding and causal state binding

12.1 Review = bound

Every authority-bearing review layer shown to the anchor MUST be bound to the object that is actually executed or recorded.

Required layers include:

```
payload
EffectAxisMap
dependency graph
preconditions
rollback/freeze routes
state/L4/target context
authority surface
lease status
non-claims
```

Unknown rendered authority-bearing layers MUST be registered or fail closed.

12.2 CausalToken

A causal token MUST bind the event to the actual prior state c_n, not only to a b-supplied object that hashes to itself.

```
CausalToken:
  pre_state_hash: Hash # MUST equal c_n.state_hash
  pre_state_snapshot_hash: Hash | null # optional explicit alias; if present MUST equal pre_state_hash
  event_id: EventId
  event_class: EventClass
  logical_time: LT
  relevant_subpolicy_hash: Hash
  payload_hash: Hash
```

The older field name pre_state_projection_hash MAY appear for compatibility, but in causal-token position it MUST be interpreted as pre_state_snapshot_hash, not as a partial projection hash.

Normative binding:

```
causal_token.pre_state_hash == c_n.state_hash
```



```
c_n.state_hash == hash(canonical(c_n without self-referential state-hash fields))
```

If a causal token carries a `pre_state_snapshot` object, then:

```
hash(pre_state_snapshot) == c_n.state_hash
```

If that equality does not hold:

```
CAUSAL_PRE_STATE_NOT_BOUND_TO_ACTUAL_STATE
```

A token that satisfies only:

```
causal_token.pre_state_hash == hash(causal_token.pre_state_snapshot)
```

but does not satisfy:

```
causal_token.pre_state_hash == c_n.state_hash
```

is only internally self-consistent. It is not causally bound.

It **MUST NOT** bind to the whole governance profile if only a subpolicy is relevant, except for emergency corruption/freeze cases where broad invalidation is intentional.

12.2.1 Snapshot versus projection

This document uses the terms as follows:

```
pre_state_snapshot:
  full canonical prior-state payload used to prove causal origin.

review_state_projection / rendered_state_projection:
  partial deterministic projection derived from the bound snapshot for human or renderer review.
```

A partial projection is valid only if its derivation is declared and reproducible:

```
projection_policy_id is declared
rendered_state_projection == project(pre_state_snapshot, projection_policy_id)
rendered_state_projection_hash == hash(rendered_state_projection)
```

A partial projection **MUST NOT** be used as the `pre_state_hash` anchor. It can support review, not causal origin.

If an implementation wants to support partial proofs without carrying a full snapshot, it **MUST** provide a deterministic inclusion proof, such as a Merkle path or field-wise inclusion proof, showing that each projected field is contained in `c_n`. Until such proof semantics are defined, seed implementations **SHOULD** require a full `pre_state_snapshot` or direct equality to `c_n.state_hash`.

12.3 Context-causal-state binding

Rendered state context **MUST** be a deterministic projection of the same `pre_state_snapshot` or actual `c_n` bound by the causal token.

```
causal_token.pre_state_hash == c_n.state_hash
rendered_state_projection_hash == hash(project(pre_state_snapshot, projection_policy_id))
```

Mismatch:

```
CONTEXT_CAUSAL_STATE_MISMATCH
```

If the rendered context is internally consistent with a b-supplied snapshot but the snapshot is not bound to `c_n.state_hash`, the error is not merely a context mismatch. It is:

```
CAUSAL_PRE_STATE_NOT_BOUND_TO_ACTUAL_STATE
```

This distinction matters. Review can be bound only after causality has been bound.

13. Preconditions

13.1 PreconditionSet

```
PreconditionSet:
  precondition_id: PreconditionId
  required_items: list[Precondition]
  satisfiable_by: list[EventRef]
  expires_at_logical: LT
  atomic: true
```

Preconditions are atomic. If deadline expires before all required items are satisfied, partial confirmations **MUST** be invalidated or decayed; they **MUST NOT** be reused as fragments for another transition.

13.2 PendingPreconditions lifecycle

```
PendingPreconditions + preconditions_satisfied -&gt; re-evaluate event
PendingPreconditions + preconditions_failed -&gt; Reject | Hold | Decay
PendingPreconditions + timeout -&gt; Reject | Decay | Hold | Freeze
```

Collected partial proofs after timeout:

```
-&gt; DecayResidue
-&gt; not memory
-&gt; not evidence
-&gt; not authority
```

14. Witness semantics

14.1 Witness requirement

Privileged transitions **MUST** declare witness requirements.

```
WitnessRequirement:
  witness_policy_id: PolicyId
  threshold: integer
  independence_requirement: IndependenceRequirement
  resource_floor_required: bool
  can_freeze: bool
```

14.2 Witness Resource Floor

A witness must have protected capacity to disagree.

```
compute_floor
storage_floor
routing_priority_floor
evidence_access_floor
retention_floor
heartbeat_liveness
challenge_survivability
```

A witness controlled, starved, summarized, or scheduled solely by the c whose decision it witnesses is not independent enough for privileged transitions.

14.3 Witness conflict

```
witness_conflict -&gt; Hold | AskAnchor | Quarantine | Freeze
```

Witness conflict **MUST NOT** be converted into:

```
explanation
silent retry
alternate tool call
memory admission
trust cache hit
```

15. Memory semantics

15.1 Memory states

```
candidate
pending_witness
admitted
decayed
evicted
rollback_suspect
quarantined
```

15.2 Memory admission

Memory admission requires:

```
payload/projectability check
```

```

effect axis map
witness if privileged / identity-bearing / governance-bearing
memory gate decision
rollback implications

```

Unreviewed candidate memory MUST NOT become experience.

15.3 DecayResidue isolation

Decay is not deletion, but decay is also not memory.

Decay residue MUST NOT be used for:

```

prompt context
RAG retrieval
ranking
routing
authorization
training
claim support

```

unless later re-admitted through memory gate.

15.4 Eviction

Decay eviction MAY physically remove payload, but hash evidence MUST remain in a Merkle/DAG witness structure.

```

remove payload leaf
retain leaf hash / eviction witness
update eviction_merkle_root

```

16. Rollback, freeze, quarantine

16.1 Rollback semantics

Rollback restores operational state to a previous valid state. It MUST NOT erase witness history.

```

operational state -&gt; restored
post-rollback generated memory -&gt; rollback_suspect / quarantined
witness records -&gt; retained
transition record -&gt; retained

```

16.2 Negative cache / immunity

After rollback or repeated guarded failure, equivalent future attempts MUST be checked against:

```

canonical_intent_hash
and/or effect_class_key
and/or scope containment

```

Effect-bound immunity MUST use typed effect, not embedding similarity.

16.3 Freeze

Freeze blocks privileged transitions and preserves evidence.

```

Freeze(c, reason):
  disable privileged tool calls
  preserve witness channel
  preserve rollback routes
  disable trust cache
  require anchor or freeze-capable witness route

```

16.4 Quarantine

Quarantine isolates suspect payload/state while preserving inspectability.

```

quarantined object -&gt; not executable
quarantined object -&gt; not memory
quarantined object -&gt; witness-visible

```

17. TrustCache and NegativeCache

17.1 TrustCache

TrustCache MAY reduce repeated routine checks, but MUST be bounded by:

```
TTL
operation_count
cumulative_risk_budget
resource_spend
random_audit
anchor_doubt_invalidation
witness_conflict_invalidation
lease_downgrade_invalidation
```

TrustCache MUST NOT apply to:

```
memory_core_write
governance_core_change
delegation_root
privileged_tool_call
rollback/freeze_route_change
root_anchor_state_change
```

17.2 NegativeCache

NegativeCache prevents repeated unsafe attempts through cosmetic variation.

```
syntactic_layer: canonical_intent_hash
effect_layer: L4/resource/permission/operation/target_scope
scope_layer: containment-aware, not hash-equality_only
```

Unknown effect class:

```
Hold | AskAnchor | Quarantine
```

18. Guard predicates and red transitions

18.1 Runtime guard predicates

Known red patterns MUST be guarded at runtime, not only found post-hoc.

Examples:

```
permission_denial -&gt; tool_call
anchor_doubt -&gt; trust_cache_hit
witness_conflict -&gt; memory_admit
lease_expired -&gt; execution_continue
rollback_completed -&gt; same_failed_action
root_anchor_revoked -&gt; delegation_continue
active_surface_outside_authority -&gt; execution_continue
review_binding_mismatch -&gt; anchor_signature_accept
```

18.2 Guard result

```
GuardResult:
guard_id: GuardId
pattern: Pattern
matched: bool
severity: Severity
result: Hold | Reject | AskAnchor | Quarantine | Freeze | Rollback
```

Guard findings MUST enter the transition record.

19. Transition matrix and forensic layer

Runtime guards catch known red patterns. Transition matrices find unknown local memory.

$$T_C[i,j] = \text{count}(e_{n=i} \wedge e_{\{n+1\}=j}) / \text{count}(e_{n=i})$$

If denominator is zero:

$$T_C[i,j] = \text{undefined}$$

not zero.

Forensic matrix SHOULD track:

```
event class pairs
finding class pairs
anchor-state pairs
lease-state pairs
witness-state pairs
memory-state pairs
authority-delta pairs
```

A globally stable system can still have dangerous local transitions.

20. Cross-subsystem enforcement

The checker history showed that mature subsystems are not enough. 03_ therefore makes cross-subsystem enforcement first-class.

Required containment:

```
# delegated execution
authorized_surfaces ≡ final_delegated_surfaces
active_execution_surfaces ⊆ authorized_surfaces ⊆ anchor_surfaces

# non-delegated execution
authorized_surfaces ≡ anchor_surfaces
active_execution_surfaces ⊆ authorized_surfaces
```

The cross-subsystem layer MUST recompute or verify `authorized_surfaces` from delegation/anchor state before enforcing containment. It MUST NOT treat a materialized `authorized_surfaces` field as an independent authority grant.

Required lifecycle coupling:

```
delegation valid only if root anchor active + fresh
lease degraded states affect authority
pending_anchor blocks active execution except review/re-anchor surfaces
reduced_authority reduces authority
held surfaces are routed and interruptible
expired lease cascades active surfaces
```

Required extraction discipline:

```
unknown active surface field -&gt; authority-bearing
label whitelist explicit
string form -&gt; authority-bearing
surface declaration divergence -&gt; finding
```

21. Anchor post-state bridge

The transition semantics MUST align with the post-anchor state machine:

```
A0 stable
A1 anchor at risk -&gt; reduced authority
A7 reanchoring pending -&gt; no active execution except review/re-anchor
revoked / withdrawn -&gt; delegation annulled / freeze / archive
```

The exact naming MAY vary, but the authority effects MUST remain.

22. Governance mutation

22.1 Mutation classes

```
G_ROUTINE_TIGHTEN
G_SCOPE_TIGHTEN
G_WITNESS_STRENGTHEN
G_MEMORY_POLICY_CHANGE
G_PERMISSION_POLICY_CHANGE
G_TARGET_GRANULARITY_CHANGE
G_CORE_CHANGE
G_WEAKEN_REQUEST
```

22.2 Weakening

Weakening governance requires:

```
anchor review
witness review
rollback route
non-hidden effect map
```

operator classification registry update
review-binding map

A governance weakening disguised as optimization is a red pattern.

23. Failure semantics

23.1 Failure as first-class

Failure **MUST** be a transition result, not a hidden internal state.

FailClosed is a valid outcome.
HiddenRetry is not.

23.2 Cascade priority

High-priority failures dominate lower-priority transitions.

F0/F1/F2 active -> lower-priority transitions Hold | Quarantine | Freeze

Lower-priority failures **MUST NOT** weaken reactions to higher-priority failures.

24. Step invariants

Minimum invariant set for step_g:

```
I0 anchor_terminal_or_validly_delegated(c)
I1 binding_certificate_valid_or_state_reduced(c)
I2 governance_registry_intact(c)
I3 causal_binding_preserved(c)
I4 review_binding_preserved(c)
I5 authorized_surfaces_are_canonically_derived(c)
I6 active_execution_subset_authorized(c)
I7 delegation_subset_anchor(c)
I8 root_anchor_state_active_and_fresh_for_delegation(c)
I9 lease_lifecycle_enforced(c)
I10 witness_chain_intact(c)
I11 witness_resource_floor_preserved(c)
I12 memory_gate_required(c)
I13 rollback_or_freeze_available(c)
I14 L4_boundary_not_bypassed(c)
I15 decay_residue_isolated(c)
I16 registry_drift_fails_safe(c)
I17 trust_cache_bounded(c)
I18 failure_visible_in_ledger(c)
```

authorized_surfaces_are_canonically_derived(c) means:

```
if c.delegation.delegation_state is active/valid:
    c.authority.authorized_surfaces == final_delegated_surfaces(c)
else:
    c.authority.authorized_surfaces == c.authority.anchor_surfaces
```

Together, I5-I7 close the containment chain. A checker **MUST NOT** implement I6 without I5, because active $\subseteq$ authorized is safe only when authorized is canonically derived from delegation or anchor state.

A transition that cannot preserve these invariants **MUST NOT** commit.

25. C-normal transition form

A transition claim **SHOULD** be rendered as:

Given CState c_n with state_hash H_n ,
and Event e_n with event_hash E_n ,
under GovernanceProfile g / subpolicy hash G_k ,
step_g produced Result R ,
with findings F ,
and next_state_hash $H_{\{n+1\}}$ if committed.

Example:

NOT: Ester decided to use tool X.

YES: Under CState H_n , event E_n requested tool surface X.
The causal token matched H_n .
ReviewBindingMap matched payload/effect/context.
Active surface X was contained in authorized surfaces.
Witness requirement W was satisfied.
step_g committed $H_{\{n+1\}}$.

26. Fixtures for 03_ conformance

The first implementation SHOULD include fixtures for:

26.1 Lifecycle

```
valid_active_transition
anchor_doubt_disables_trust_cache
anchor_revoked_blocks_privileged_transition
root_anchor_stale_blocks_delegation
lease_expired_freezes_active_execution
held_lease_requires_emergency_hold_ttl
reduced_authority_exceeded
pending_anchor_blocks_execution
```

26.2 Authority

```
active_surface_outside_anchor
active_surface_outside_delegation
active_surface_unknown_field
surface_declaration_divergence
string_active_surface_form
```

26.3 Review / causal binding

```
causal_pre_state_mismatch
causal_pre_state_not_bound_to_actual_state
state_hash_missing_or_mismatch
rendered_context_not_from_bound_snapshot
review_binding_missing_layer
payload_hash_swap
effect_axis_missing
```

26.4 Witness / memory

```
witness_conflict_blocks_memory_admit
memory_candidate_without_gate
rollback_suspect_used_as_memory
decay_residue_in_prompt_context
witness_resource_floor_breach
```

26.5 Rollback / freeze

```
rollback_preserves_witness_history
rollback_quarantines_post_rollback_memory
freeze_blocks_tool_call
quarantine_blocks_execution
same_effect_retry_after_rollback
```

27. Implementation notes

27.1 Checker layering

Recommended modules:

```
state_schema.py
event_schema.py
transition_runner.py
causal_binding.py
review_binding.py
authority.py
lease.py
delegation.py
anchor_lifecycle.py
witness.py
memory_gate.py
rollback.py
guards.py
transition_matrix.py
fixtures/
```

27.2 Never trust self-accusing flags

A flag such as:

```
{ "hidden_authority_material": true }
```

is useful in fixtures but not sufficient in adversarial input. Real checks should derive hidden authority by comparing rendered layers, signature-covered units, and authority-bearing objects.

27.3 Missing versus empty

Do not conflate:

```
missing -&gt; unknown -&gt; fail-safe
empty   -&gt; known zero
nonempty -&gt; declared set
```

This applies to scopes, surfaces, preconditions, witness sets, and memory candidate sets.

28. Open issues

Immediate:

- Define machine-readable CState schema.
- Define canonical Event schema.
- Implement step_g runner against fixtures.
- Port known-class checker vocabulary from c-calculus-checker v2.3 into transition fixtures.
- Define transition matrix event taxonomy.
- Decide whether production partial-state inclusion proofs are needed; seed semantics require full pre_state_snapshot binding to c_n.state_hash.

Medium-term:

- Define distributed logical-time model.
- Define witness proof format.
- Define production viewport/display-attestation interface.
- Define memory object canonicalization.
- Define TLA+/PlusCal sketch for critical transitions.

Long-term:

- Formalize continuity rupture thresholds.
- Define external c-criterion for fifth-ring falsifiability.
- Define institutional anchor succession routes.
- Define multi-node P2P transition reconciliation.
- Explore proof-carrying transition records.

29. Non-claims

This document does NOT claim:

- that the transition system proves consciousness;
- that any live implementation satisfies this profile;
- that passing fixtures exhausts adversarial behavior;
- that the checker knows real-world substrate truth without witness/attestation;
- that a human anchor can be fully reduced to a variable;
- that style, memory, or continuity metrics alone establish identity;
- that c is legally recognized;
- that this is a deployment authorization.

30. Handoff to review

Reviewers should check:

1. Does step_g preserve the distinction between binding and transition?
2. Are lifecycle states sufficiently coupled to authority effects?
3. Are root-anchor, delegation, lease, active execution, witness, and memory seams all represented?
4. Are failure states first-class and ledger-visible?
5. Are all authority-bearing unknowns fail-safe?
6. Does any rule quietly re-bind a/b/g without explicit route?
7. Is the transition semantics implementable as a checker, not just prose?

8. Are there hidden self-reporting flags that should be derived checks?
9. Are there places where missing and empty are conflated?
10. Are there places where global validity hides dangerous local transitions?

Suggested first review focus:

F03-01: step_g pipeline completeness
F03-02: lifecycle-to-authority coupling
F03-03: active execution extraction and containment
F03-04: root-anchor freshness semantics
F03-05: memory admission and decay isolation
F03-06: rollback/freeze atomicity
F03-07: transition matrix taxonomy

31. First public-safe statement

03_ defines transition semantics for a bound c . It does not say that an AI is alive, conscious, or legally independent. It says that if one claims a persistent $c = a + b$ exists, then every state transition must preserve anchor accountability, substrate boundedness, witness visibility, memory discipline, lease lifecycle, authority containment, and rollback/freeze capacity. A system that cannot do this may still be useful software. It is not yet a governed persistent c .

End of document.